Lecture 28.06: Information Gain & Split Selection Mechanics

📌 Executive Summary

Quantifying Information Gain as mutual information, weighted child impurity reduction, C4.5 Gain Ratio normalization against high-cardinality bias, and worked split selection.

🏛️ Theoretical Formulation & Mathematical Architecture

💡 Information Gain Mathematical Formulation

Given a parent node dataset S containing $|S|$ samples and a proposed binary split on feature A yielding left child S_L and right child S_R:

$$\boxed{IG(S, A) = I(S) - \left(\frac{|S_L|}{|S|} I(S_L) + \frac{|S_R|}{|S|} I(S_R) \right)}$$

Where $I(\cdot)$ denotes the chosen impurity metric (either Entropy H or Gini G).

In information theory, $IG(S, A)$ is equivalent to the **Mutual Information** $I(Y; A)$ between the supervisory target Y and the split attribute A.

🛡️ High-Cardinality Pathology & C4.5 Gain Ratio

If an attribute A possesses many distinct values (e.g., `User_Session_ID`), pure Information Gain will partition the data into $|S|$ singleton leaf nodes:

- Each child node has $|S_v| = 1 \implies I(S_v) = 0$.
- The weighted child impurity drops to 0.0.
- Information Gain reaches its absolute maximum $IG = I(S)$, causing catastrophic overfitting.

Ross Quinlan's **C4.5 Algorithm** rectifies this pathology by normalizing $IG(S, A)$ by the **Split Information**:

$$SplitInfo(S, A) = -\sum_{v=1}^{V} \frac{|S_v|}{|S|} \log_2 \left(\frac{|S_v|}{|S|} \right)$$

The normalized criterion is the **Gain Ratio**:

$$\boxed{GainRatio(S, A) = \frac{IG(S, A)}{SplitInfo(S, A)}}$$

Split Information acts as a penalizing regularizer: splits that generate a multitude of uniformly sized small buckets suffer a high $SplitInfo$ denominator, driving the Gain Ratio down.

💻 Vectorized Python Implementation

```
import numpy as np

def compute_information_gain(parent_y, left_y, right_y, metric_fn):
    n_p = len(parent_y)
    n_l = len(left_y)
    n_r = len(right_y)
    if n_p == 0 or n_l == 0 or n_r == 0:
        return 0.0

    parent_imp = metric_fn(parent_y)
    weighted_child_imp = (n_l / n_p) * metric_fn(left_y) + (n_r / n_p) * metric_fr
    return parent_imp - weighted_child_imp

def gini(labels):
    _, c = np.unique(labels, return_counts=True)
    return 1.0 - np.sum((c / c.sum())**2)

# Test
p_labels = np.array([1, 1, 1, 1, 0, 0, 0, 0]) # 50-50 mix
l_labels = np.array([1, 1, 1, 1])             # perfectly pure
r_labels = np.array([0, 0, 0, 0])             # perfectly pure

ig = compute_information_gain(p_labels, l_labels, r_labels, gini)
print("Information Gain for perfect split:", ig)
assert ig == 0.5, "Perfect binary split of 50-50 mix must yield IG = 0.5 (Gini)"
```

🔬 Key Engineering Takeaways & Invariants

1. **Axis-Aligned Partitioning**: Decision trees recursively split feature space along orthogonal boundaries without requiring feature scaling.
2. **Impurity Optimization**: Split selection deterministically maximizes impurity reduction (Information Gain for classification, Variance Reduction for regression).
3. **Generalization Safeguards**: Regularization via pre-pruning (`max_depth` , `min_samples_leaf`) and post-pruning (`ccp_alpha`) prevents noisy leaf memorization.